



Software Analyzers

E-ACSL User Manual





FRAMA-C's E-ACSL Plug-in

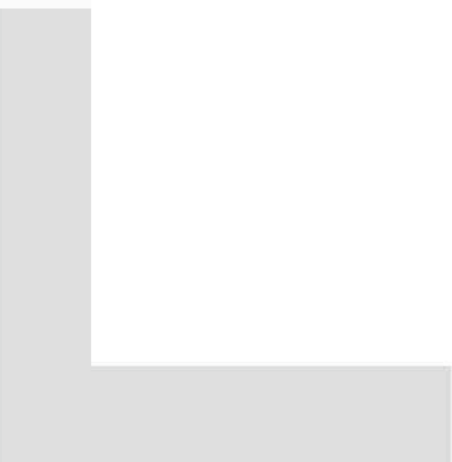
Release 0.1+dev compatible with Fluorine-20130501

Julien Signoles

CEA LIST, Software Safety Laboratory, Saclay, F-91191

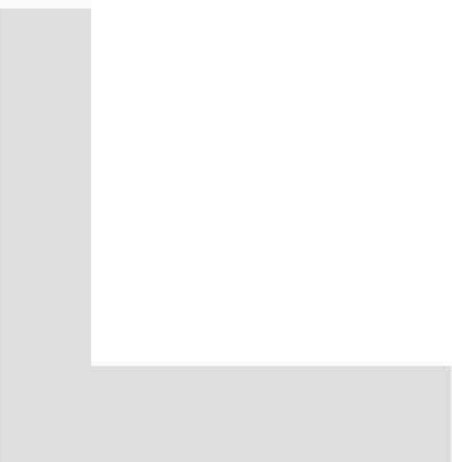
©2013 CEA LIST

This work has been supported by the 'Hi-Lite' FUI project (FUI AAP 9).



Contents

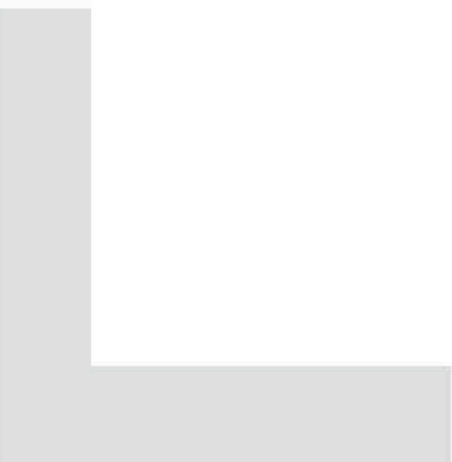
Foreword	7
1 Introduction	9
2 What the Plug-in Provides	11
2.1 Simple Example	11
2.2 Customizing the Plug-in	13
2.3 Execution Environment of the Generated Code	13
2.4 Potential Runtime Errors in the Generated Code	13
2.5 Handling Incomplete Program	13
2.6 Combining E-ACSL with Others Plug-ins	14
2.7 Verbosing Policy	14
3 Known limitations	15
Bibliography	17
Index	19



Foreword

This is the user manual of the FRAMA-C plug-in E-ACSL¹. The content of this document corresponds to its version 0.1+dev (May 16, 2013) compatible with the version Fluorine-20130501 of FRAMA-C [3, 5]. However the development of the E-ACSL plug-in is still ongoing: features described here may still evolve in the future.

¹<http://frama-c.com/eacsl>



Chapter 1

Introduction

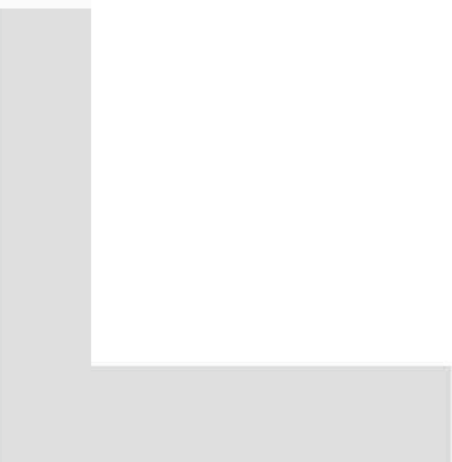
FRAMA-C [3, 5] is a modular analysis framework for the C language which supports the ACSL specification language [1]. This manual documents the E-ACSL plug-in of FRAMA-C. This plug-in automatically translates an annotated C code into a C code which fails at runtime if an annotation is violated. If no annotation is violated, the behavior of the new program is exactly the same than the one of the original program.

Such a translation brings several benefits. First it allows the user to monitor a C code, in particular to perform what is usually called runtime assertion checking [2]¹. That is the primary goal of E-ACSL. Second it allows to combine FRAMA-C and its existing analyzers, with other analyzer tools for C, even those who do not understand the ACSL specification language. Third, the possibility to detect invalid annotations during a concrete execution may be very helpful while writing a correct specification of a given program, *e.g.* for later program proving. Finally, an executable specification makes it possible to check runtime assertions that cannot be verified statically and to establish a link between monitoring tools and static analysis tools like VALUE [6] or WP [4].

Annotations must be written in the E-ACSL specification language [8, 7] which is a subset of ACSL. This plug-in is still in a preliminary state: some parts of E-ACSL are not yet supported. Which E-ACSL annotations are currently handled by the E-ACSL plug-in is documented in a separated document [9].

This manual does ***not*** explain how to install the plug-in. Please have a look at file `INSTALL` of the E-ACSL tarball for this purpose.

¹In our context, “runtime annotation checking” would be a better more-general expression.



Chapter 2

What the Plug-in Provides

This chapter is the core of this manual and describes how to use the plug-in. First, Section 2.1 shows how to run the plug-in on a trivial example and how to execute the generated code with a standard C compiler to detect invalid annotations at runtime. Then, Section 2.2 introduces how to customize the plug-in. Next, Section 2.3 provides additional details on the execution of the generated code. Section 2.4 explains how the plug-in handles potential runtime errors in the generated code, while Section 2.5 focus on how to deal with incomplete programs, *i.e.* in which some functions have no body or in which there are no main function. After, Section 2.6 explains how to combine the plug-in with other plug-ins of FRAMA-C. Finally, Section 2.7 details the verbosing policy of the plug-in.

2.1 Simple Example

The main option of the plug-in is `-e-acsl`. Consider the following program.

File `first.i`

```
int main(void) {
    int x = 0;
    /*@ assert x == 0; */
    /*@ assert x == 1; */
    return 0;
}
```

Running E-ACSL on this file is then pretty simple:

```
$ frama-c -e-acsl first.i
[kernel] preprocessing with <...> share/frama-c/e-acsl/e_acsl_gmp_types.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/e_acsl_gmp.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/e_acsl.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/memory_model/e_acsl_mmodel_api.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/memory_model/e_acsl_bittree.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/memory_model/e_acsl_mmodel.h
[e-acsl] beginning translation.
[e-acsl] translation done in project "e-acsl".
```

Even if `first.i` is already preprocessed, E-ACSL first asks the FRAMA-C kernel to preprocess and link with `first.i` several files which forms the so-called E-ACSL library. Its usefulness will be explain later, mainly in Section 2.3.

Then E-ACSL takes the annotated C code as input and translates it into a new FRAMA-C project named `e-acsl`¹. By default, the option `-eacsl` does nothing more. It is however

¹The notion of *project* is explained in Section 8.1 of the FRAMA-C user manual [3].

possible to have a look at the generated code on the FRAMA-C GUI. It is also possible through the command line thanks to the kernel options `-then-on` and `-print`:

```
$ frama-c -e-acsl first.i -then-on e-acsl -print
[kernel] preprocessing with <...> share/frama-c/e-acsl/e_acsl_gmp_types.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/e_acsl_gmp.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/e_acsl.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/memory_model/e_acsl_mmodel_api.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/memory_model/e_acsl_bittree.h
[kernel] preprocessing with <...> share/frama-c/e-acsl/memory_model/e_acsl_mmodel.h
[e-acsl] beginning translation.
[e-acsl] translation done in project "e-acsl".
/* Generated by Frama-C */
struct __anonstruct__mpz_struct_1 {
    int _mp_alloc ;
    int _mp_size ;
    unsigned long *_mp_d ;
};
typedef struct __anonstruct__mpz_struct_1 __mpz_struct;
typedef __mpz_struct ( __attribute__((__FC_BUILTIN__)) mpz_t)[1];
typedef unsigned int size_t;
/*@
model __mpz_struct { integer n };
*/
/*@ requires predicate != 0;
    assigns \nothing; */
extern __attribute__((__FC_BUILTIN__)) void e_acsl_assert(int predicate,
                                                         char *kind,
                                                         char *fct,
                                                         char *pred_txt,
                                                         int line);

int __fc_random_counter __attribute__((__unused__));
unsigned long const __fc_rand_max = (unsigned long)32767;
/*@ ghost extern int __fc_heap_status; */

/*@
axiomatic
dynamic_allocation {
    predicate is_allocable{L}(size_t n)
    reads __fc_heap_status;

}
*/
extern __attribute__((__FC_BUILTIN__)) void __clean(void);

extern size_t __memory_size;

/*@
predicate diffSize{L1, L2}(integer i) =
    \at(__memory_size, L1) - \at(__memory_size, L2) == i;
*/
int main(void)
{
    int __retres;
    int x;
    x = 0;
    /*@ assert x == 0; */
    e_acsl_assert(x == 0, (char *)"Assertion", (char *)"main", (char *)"x == 0", 3);
    /*@ assert x == 1; */
    e_acsl_assert(x == 1, (char *)"Assertion", (char *)"main", (char *)"x == 1", 4);
    __retres = 0;
    __clean();
    return __retres;
}
```

As you can see, the generated code contains additional type declarations, constant declarations and global ACSL annotations that are not in the initial file `first.i`. That is a part of the included E-ACSL library. You can safely ignore it right now. The translated `main` function

of `first.i` is displayed at the end. Two lines have been added. The first one is just after the unique E-ACSL annotation.

```
/*@ assert x == 0; */
e_acsl_assert(x == 0, (char *)"Assertion", (char *)"main", (char *)"x == 0", 3);
```

It is a function call to `e_acsl_assert` from the E-ACSL library. This call performs the dynamic verification that the corresponding assertion is valid. More precisely, it checks that its first argument (here `x == 0` corresponding to the annotation) is not equal to 0 and fails otherwise. The extra arguments are only used to display nice user feedback as shown later in Section 2.3.

The second additional line is the call to `__clean` (once again from the E-ACSL library) just before the `return` statement. This call is required to clean potential extra allocations done by the E-ACSL library (actually none in this example).

By using the option `-ocode` of FRAMA-C, we can redirect the generate code into a C file as follows.

```
$ frama-c -e-acsl first.i -then-on e-acsl -print -ocode monitored_first.i

$ gcc -o monitored_first frama-c -print-share-path/e-acsl/e_acsl.c
monitored_first.i
$ ./monitored_first
Assertion failed at line 4 of function main.
The failing predicate is:
x == 1.
```

2.2 Customizing the Plug-in

- `-e-acsl-project`
- `-e-acsl-check`
- `-eacsl-share`

2.3 Execution Environment of the Generated Code

- takes care of architecture (32, 64 bits)
- memory model (linking issue)
- GMP (linking issue)
- customizing `e_acsl_assert`

2.4 Potential Runtime Errors in the Generated Code

- runtime error in annotations

2.5 Handling Incomplete Program

- function without code
- program without main

2.6 Combining E-ACSL with Others Plug-ins

- -e-acsl-valid
- -e-acsl-prepare

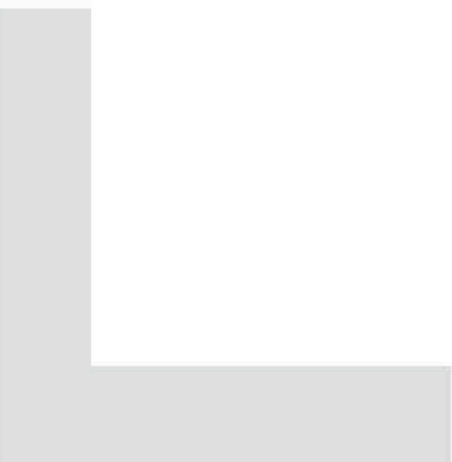
2.7 Verbosing Policy

- level
- category

Chapter 3

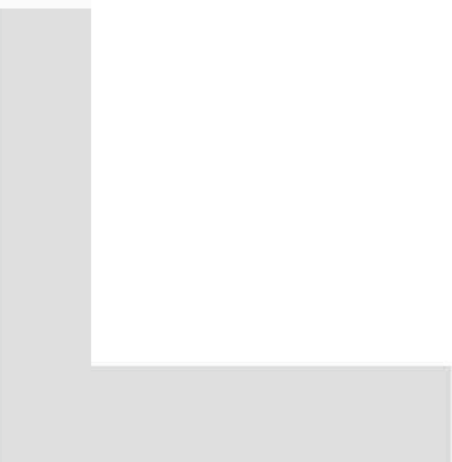
Known limitations

- of course, annotations that are not yet translated [\[9\]](#)
- function without code
- recursive functions
- function pointers
- complex control flow graph
- read `KNOWN_BUGS`



Bibliography

- [1] Patrick Baudin, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. *ACSL: ANSI/ISO C Specification Language. Version 1.7*, April 2013.
- [2] Lori A. Clarke and David S. Rosenblum. A historical perspective on runtime assertion checking in software development. *ACM SIGSOFT Software Engineering Notes*, 31(3):25–37, 2006.
- [3] Loïc Correnson, Pascal Cuoq, Florent Kirchner, Armand Puccetti, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. *Frama-C User Manual*, May April. <http://frama-c.cea.fr/download/user-manual.pdf>.
- [4] Loïc Correnson, Zaynah Dargaye, and Anne Pacalet. *Frama-C's WP plug-in*, April 2013. <http://frama-c.com/download/frama-c-wp-manual.pdf>.
- [5] Pascal Cuoq, Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-C, A software Analysis Perspective. In *Software Engineering and Formal Methods (SEFM)*, October 2012.
- [6] Pascal Cuoq, Boris Yakobowski, and Virgile Prevosto. *Frama-C's value analysis plug-in*, April 2013. <http://frama-c.cea.fr/download/value-analysis.pdf>.
- [7] Mickaël Delahaye, Nikolai Kosmatov, and Julien Signoles. Common specification language for static and dynamic analysis of C programs. In *the 28th Annual ACM Symposium on Applied Computing (SAC)*, pages 1230–1235. ACM, March 2013.
- [8] Julien Signoles. *E-ACSL: Executable ANSI/ISO C Specification Language. Version 1.7*, May 2013. URL: <http://frama-c.com/download/e-acsl/e-acsl.pdf>.
- [9] Julien Signoles. *E-ACSL Version 1.7. Implementation in Frama-C Plug-in E-ACSL version 0.2*, May 2013. URL: <http://frama-c.com/download/e-acsl/e-acsl-implementation.pdf>.



Index

`--clean`, [13](#)
`-e-acsl`, [11](#)
`e_acsl_assert`, [13](#)
`-eacsl`, [11](#)
Installation, [9](#)
`-ocode`, [13](#)
`-print`, [12](#)
`-then-on`, [12](#)